BHoardMate

Final Project Documentation

An Android boarding house management application

SE2142 — Software Engineering Tools and Practices

Section BSSE-2 • Group BJ

Bryan Kenth F. Estoesta
Screens, branding, build evidence and documentation coordination

Johnmarc B. Jayme
Data layer, state, API, navigation and technical implementation

Submission date: October 29, 2026

2. Member Roles

BHoardMate was developed by Group BJ, composed of Bryan Kenth F. Estoesta and Johnmarc B. Jayme. The group divided the work between the application interface and the supporting data and state management components. Each member remained responsible for understanding how these parts work together.

Member	GitHub account	Responsibilities
Bryan Kenth F. Estoesta	BryWick	Announcements, Concerns, Members, member details and Rooms screens; validation on his assigned screens; application icon, splash and name; APK build evidence; README and documentation coordination.
Johnmarc B. Jayme	ohimeee	Data models, Firebase services and access rules, providers, holiday API, due-date adjustment, Login, Home and Payments screens, navigation and reusable widgets; the final Technical Implementation explanation.

Both members participated in version control, pull request review and testing. Feature branches separated development work, while pull requests provided a place to discuss changes before merging them into the main branch. The group also used AI tools for assistance during development and documentation [12, 13].

3. Problem Statement

This project addresses the difficulty of keeping boarding house bills, announcements, room information, and organized maintenance reports for the boarding house owners. The original proposal identified this problem in the context of Bryan's family business. The intended users are boarding house owners and the boarders who need access to their bills and relevant notices. [3]

When records are kept on paper, owners must prepare bill breakdowns, check payment status, and maintain contact information separately. Calculating shared electricity charges with a calculator also requires the owner to keep track of consumption, the electricity rate, and the number of boarders included in the calculation. A boarder may need to ask for the same information again when the breakdown is unavailable to them.

Announcements sent through group chats can become difficult to find among unrelated messages. Maintenance reports may also lack a clear status after they have been sent. These arrangements make it harder to see which bills remain unpaid, which notices are current, and which concerns still need attention.

BHoardMate brings these records into an Android application. The project aims to make information easier to access and maintain.

4. Application Overview

BHoardMate is a Flutter application for managing the routine records and communication of a boarding house. It provides different actions for owners and boarders alike. Owners sign in using an email address and password. Boarders use a Room ID and access code supplied for their room. Firebase Authentication supports these sessions, and Cloud Firestore stores the shared application records.

The Home screen introduces the main features and displays billing and announcement summaries. Payments provide a bill breakdown containing rent, electricity, water, a total amount, and a due date. Owners can create, edit, and remove bills and change their paid status. Boarders can view the bills associated with their own room. An electricity solver assists with dividing an electricity charge among boarders.

The News tab contains announcements and Philippine public holidays. Concerns tab allows boarders to report problems for their room and owners to track whether they are pending or done. Members tab gives directory and profile details. Boarders can view fellow boardmates and edit their own information; room access codes are excluded from profiles. The owner also has room management actions for rent and access codes.

The public source repository is available at https://github.com/ohimeee/bhoardmate. It contains the application source, project history, README, and supporting evidence. Version 1.0.1 is published as a GitHub release with the installable APK file.

5. Proposed Solution

The proposed solution gives owners and boarders a common place to access information while limiting actions according to their roles. After signing in, a user can move between Home, Pay, News, and Me through the bottom navigation bar. Other features, such as Concerns and the Members directory, open from Home. Owners can additionally open Rooms.

The billing workflow begins when the owner records a room's charges and due date. A boarder can then inspect the breakdown and payment status for that room. Announcements provide a separate place for notices, while the Concerns list keeps maintenance reports visible with a status. When a tenant leaves, the owner can reset the room's access code and clear the previous boarder's profile information.

Provider supplies shared state to the screens, and the service layer handles database and API requests. This arrangement allows screens to respond to changes without placing database operations inside interface code. Firebase access rules support the distinction between owner actions and boarder permissions.

The Nager.Date API supplies Philippine public holidays. Loaded holidays are also used when displaying adjusted bill due dates. The application can show saved holiday data after a failed request. Clear form errors, loading indicators, empty states, and confirmation dialogs help users understand the result of their actions.

6. Goals and Objectives

The main goal is to develop an Android prototype that organizes boarding house records and communication for owners and boarders. The specific objectives are to:

1. Provide owner and boarder sign-in flows and display the information and actions appropriate to each role.
2. Support complete bill creation, viewing, editing, and deletion, with itemized charges, an electricity solver, and paid or unpaid status.
3. Provide announcements, concern tracking, a member directory, and owner room management within a consistent navigation structure.
4. Consume Philippine holiday data through an HTTP request, parse it into a Dart model, and show loading, success, and failure states with a saved-data fallback.
5. Apply Provider, reusable widgets, and a separate service layer so shared state and data operations remain organized.
6. Prepare a named application with a custom icon, splash, release APK, and documented development evidence. [1, 2]

7. Features

The following figures document the implemented interface for owners and boarders. Screenshots were taken on the Medium Phone API 36.1 emulator. [1]

7.1 Application identity and startup

The Android launcher displays the BHoardMate icon and name. A green native splash screen appears while Flutter starts, followed by the application's animated splash name and login view. The wordmark on Login matches the final animation frame. Native resources are configured through pubspec.yaml [19, 20]. The Flutter sequence is implemented in SplashScreen and BHoardMateSplash.

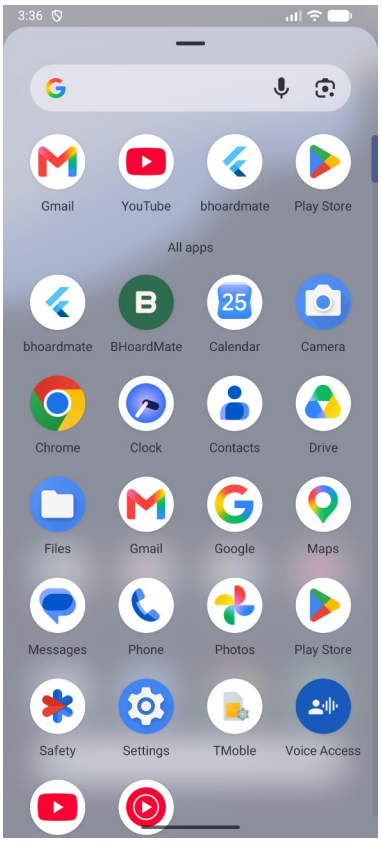


Figure 1. Launcher icon and application name, October 25.



Figure 2. Native Android splash, October 25.



Figure 3. Flutter animated splash, October 25.

7.2 Owner and boarder login

LoginScreen provides separate owner and boarder forms. Owners supply an email and password; boarders supply their room identifier and access code. Field validation and authentication errors are shown to the user. A recognized Firebase session can bypass the form and open MainScreen.

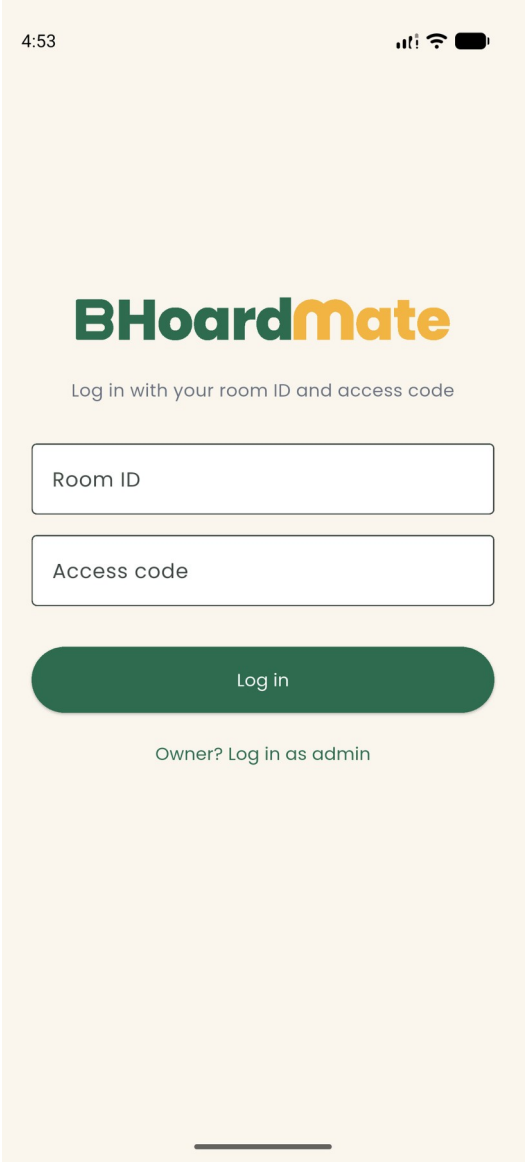


Figure 4. Boarder room-ID/access-code form, October 26.

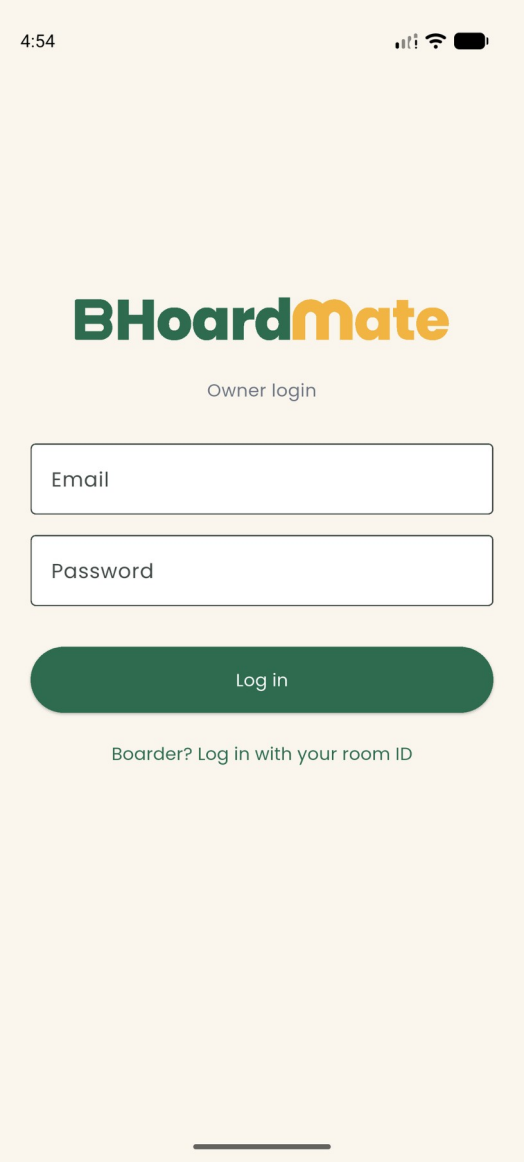


Figure 5. Owner email/password form, October 26.

7.3 Home and bottom navigation

HomeScreen summarizes outstanding charges and recent announcements and opens the main features. The owner additionally has a monthly rent-collected summary. MainScreen supplies the Home, Pay, News, and Me bottom-navigation shell visible in these figures. Its IndexedStack retains each tab while the selected index changes. The system Back button returns to Home from the other tabs, and on Home it asks the user to press Back again before the app closes.

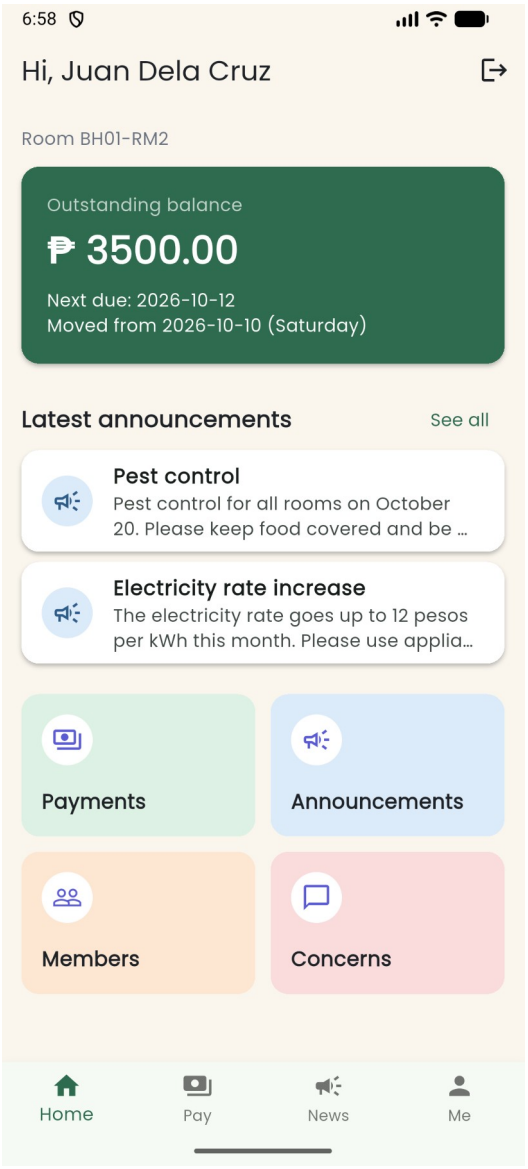


Figure 6. Installed release: boarder Home and bottom tabs, October 26.

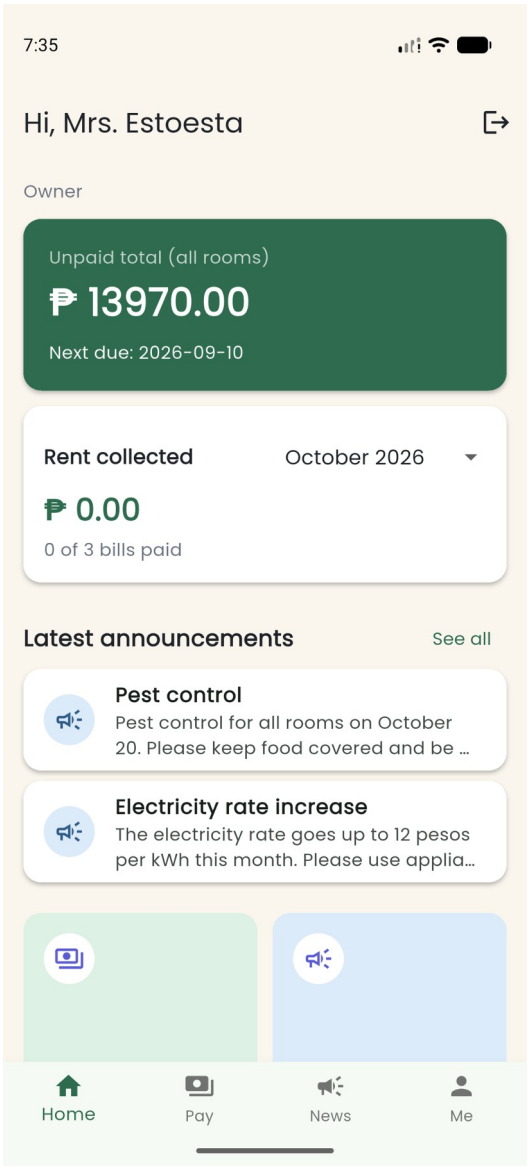


Figure 7. Owner Home summary, October 25.

7.4 Payment records and itemized bills

PaymentsScreen lists rent, electricity, and water charges, the total, due date, and payment status. Owners can add, edit, delete, and mark records paid or unpaid. Boarders read the bills associated with their own room. The provider supplies payment state to both Payments and Home, allowing the dashboard to use the same underlying records.

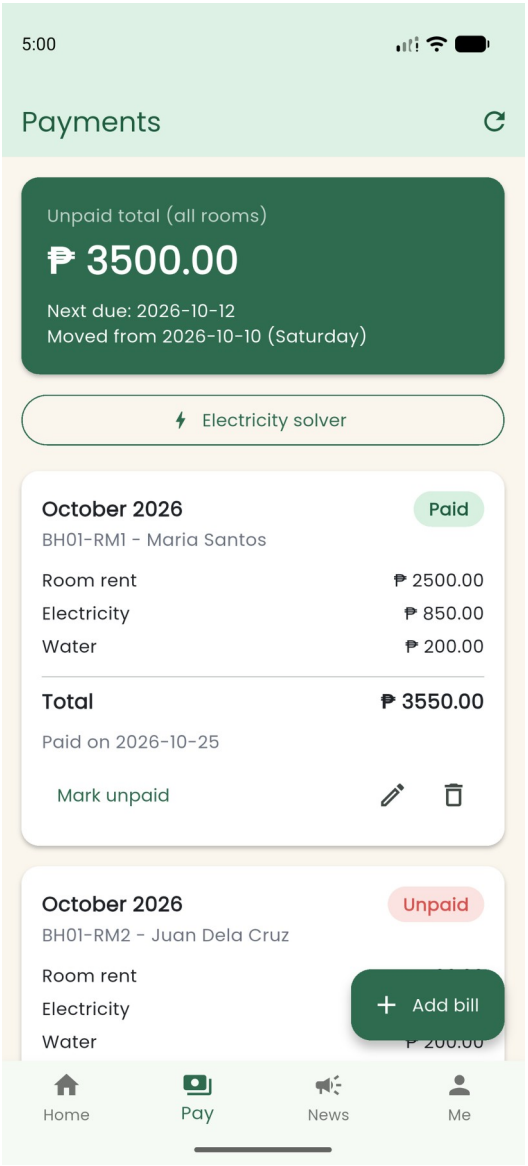


Figure 8. Owner payment controls and bills, October 26.

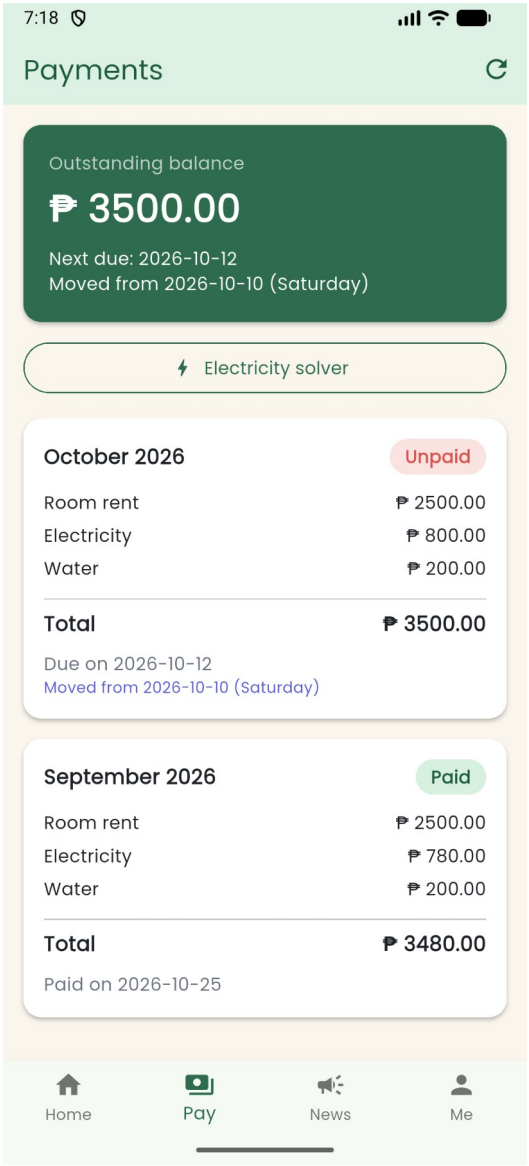


Figure 9. Installed release: boarder bill breakdown, October 26.

7.5 Add/edit bill and electricity solver

PaymentFormScreen is shared by adding and editing a bill. Its optional Payment constructor argument identifies edit mode and supplies existing values. The captured form shows adding a bill and the due-date hint. The electricity solver computes consumption × rate ÷ number of boarders; it requires valid, finite numeric inputs.

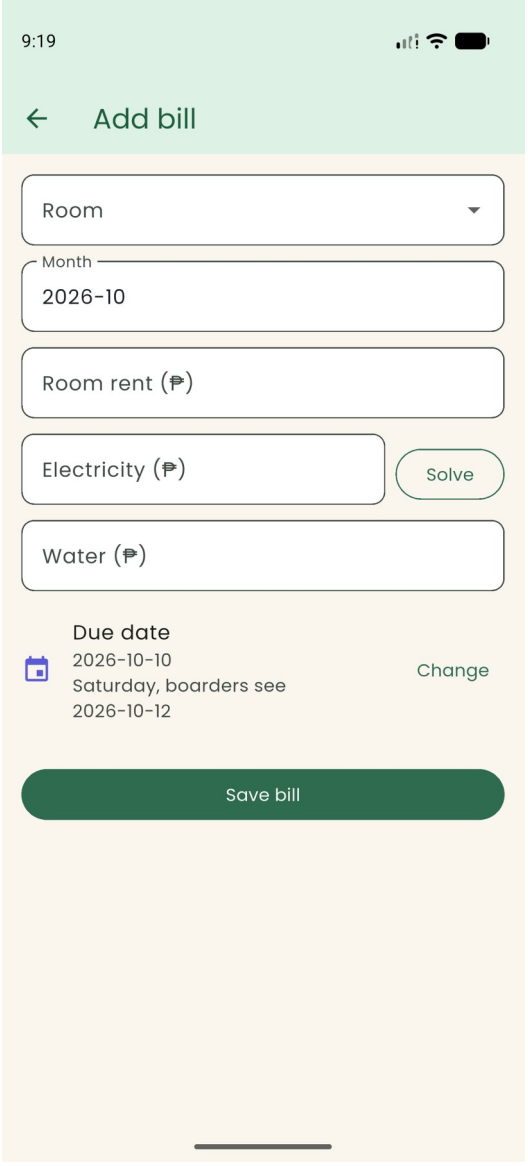


Figure 10. Add-bill form and displayed due-date hint, October 25.

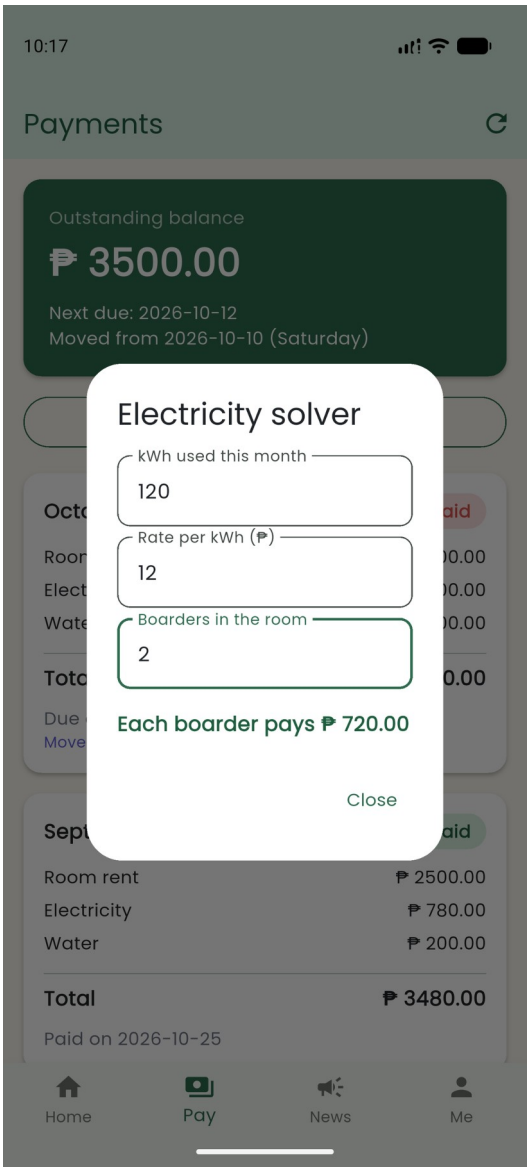


Figure 11. Electricity-splitting dialog, October 25.

7.6 News and announcements

AnnouncementsScreen is the News tab. Owners can create and edit notices and confirm removal before deleting one; boarders read notices. AnnouncementProvider supplies the current list. HolidaySection appears alongside announcements and remains available when the announcement list is empty or fails. The boarder figure was captured from the installed release.

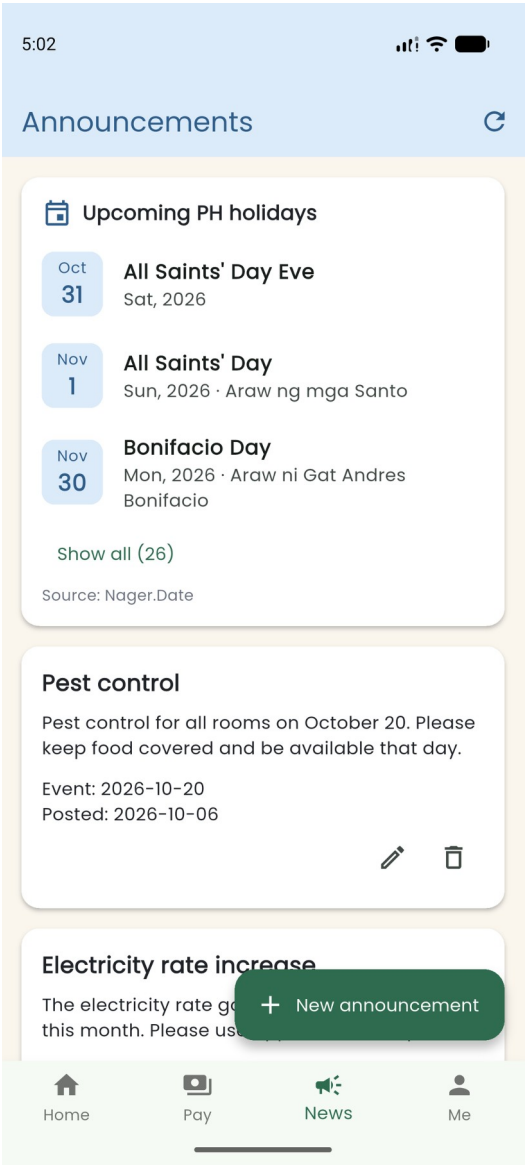


Figure 12. Owner announcements and management actions, October 26.

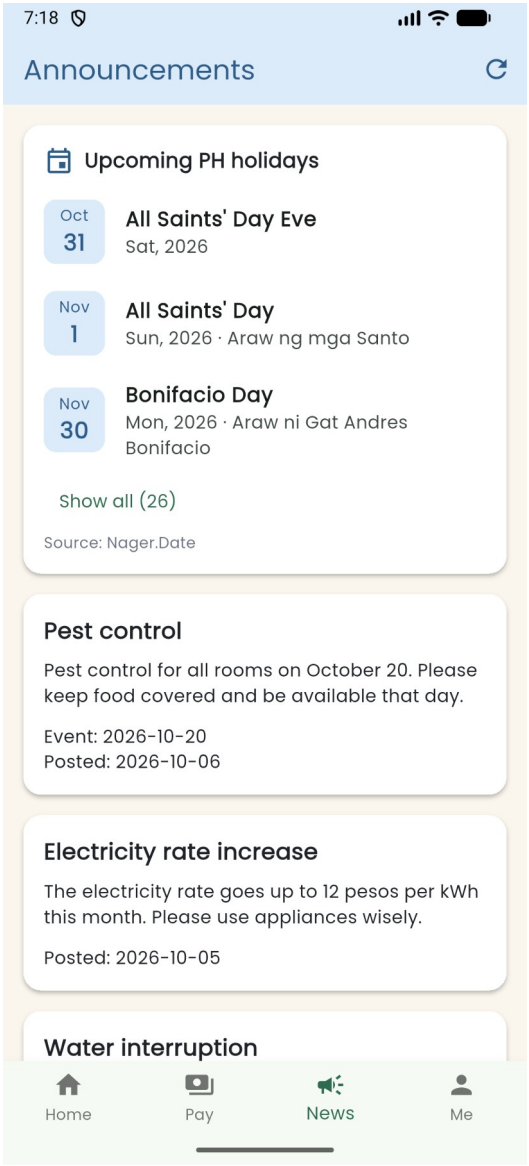


Figure 13. Installed release: boarder News and holidays, October 26.

7.7 Maintenance concerns

Boarders report concerns for their own room using a title and description. The owner can inspect concerns from all rooms and change their status between pending and done. The list uses the shared StatusChip. Removing a record requires confirmation. ConcernProvider performs the actions through the database service; the screen contains no direct Firestore calls.

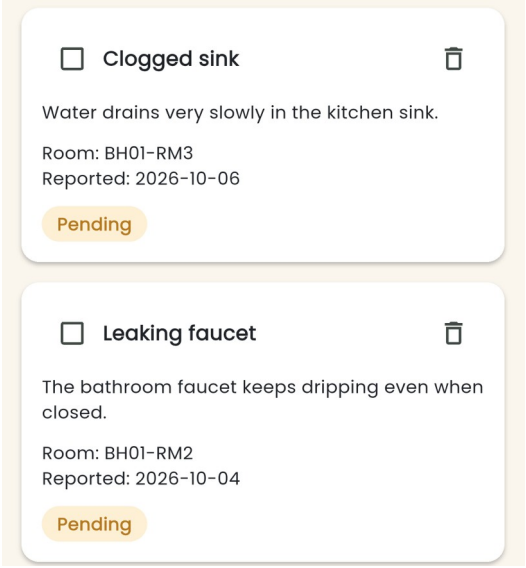


Figure 14. Owner concern list: maintenance entries and status controls (cropped), October 26.

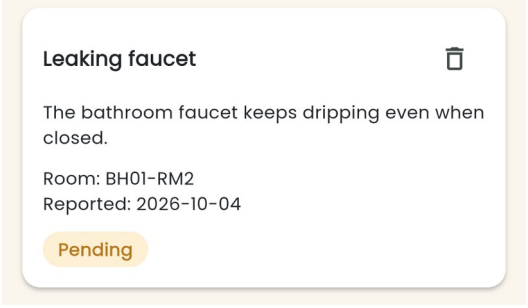


Figure 15. Boarder concern list: own-room maintenance entry (cropped), October 26.

7.8 Members directory

MembersScreen has a directory mode separate from the Me tab. Boarders can browse fellow boardmates and open their profiles in read-only mode; owners see the room/member list. Selecting a record passes its Room to MemberDetailScreen through the constructor. This provides a concrete example of information passed between screens.

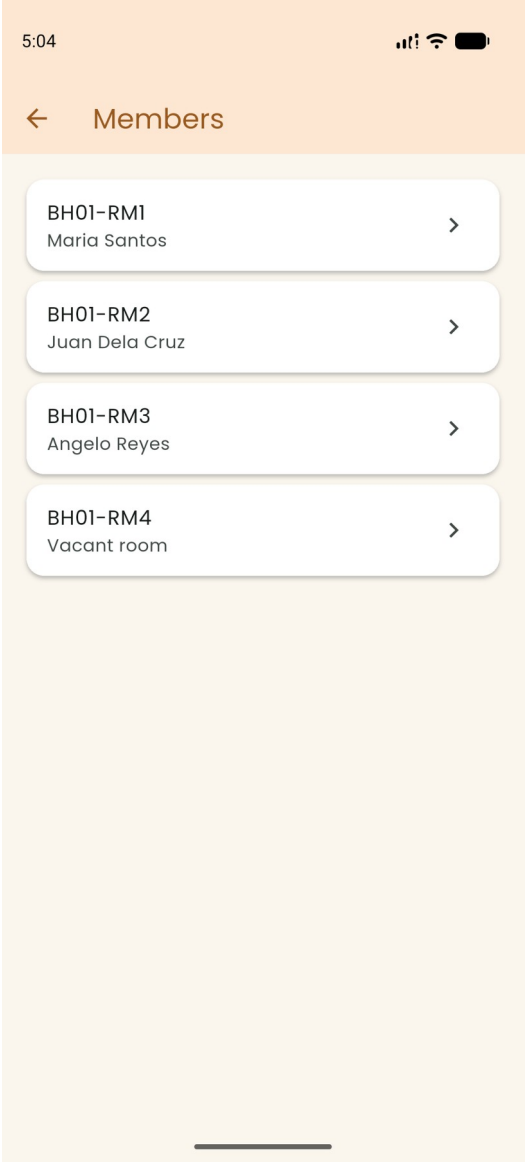


Figure 16. Owner directory, October 26.

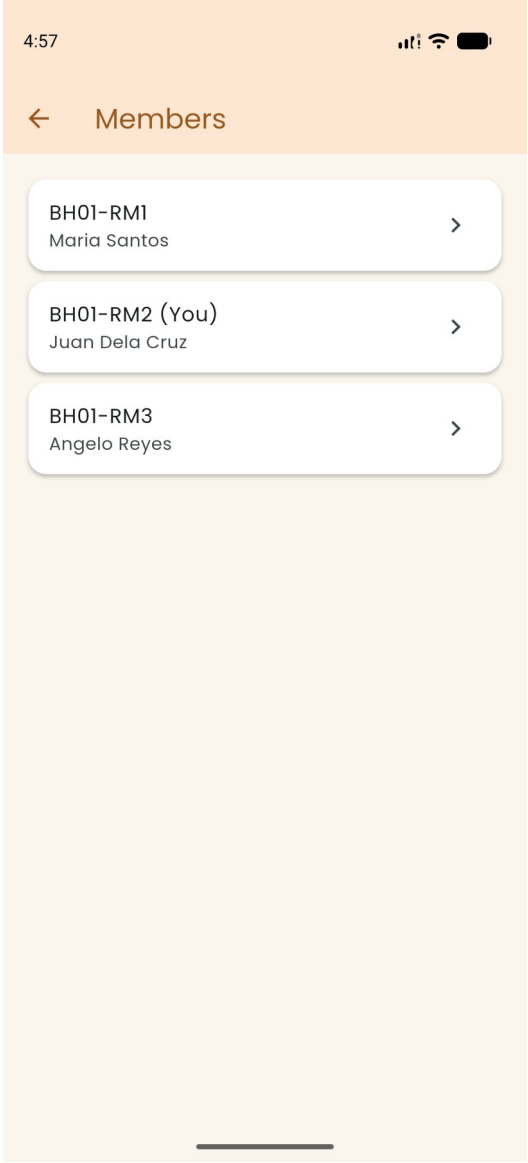


Figure 17. Boarder directory, October 26.

7.9 Boarder profile and read-only boardmate detail

The Me tab opens the signed-in boarder's own profile, where permitted profile information can be edited. Opening another boardmate's record provides a read-only view and no profile-edit action. Access codes are excluded from profile screens. The current UI also hides another boardmate's rent, age, and emergency details.

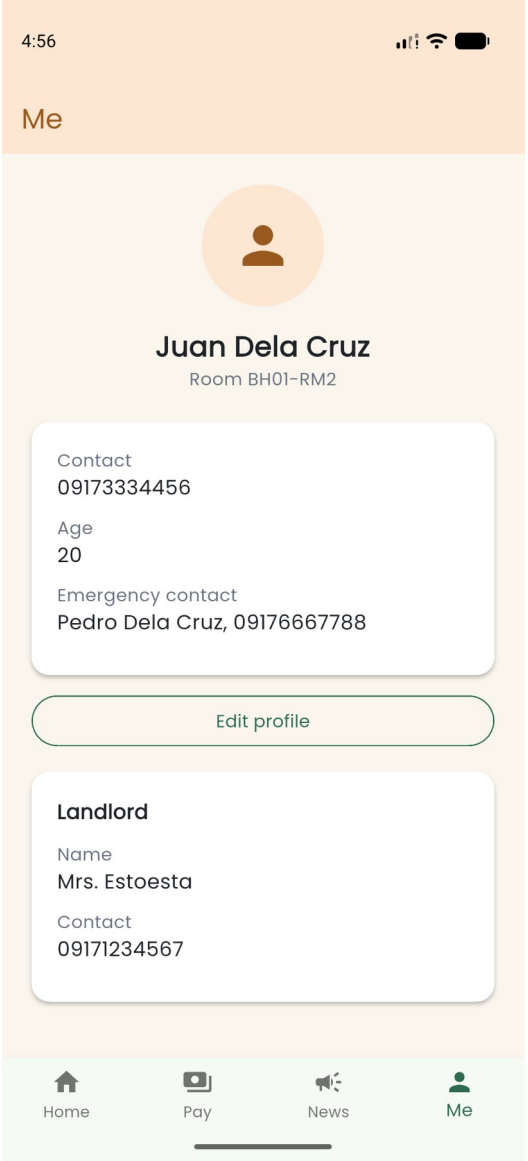


Figure 18. Boarder's own Me profile, October 26.

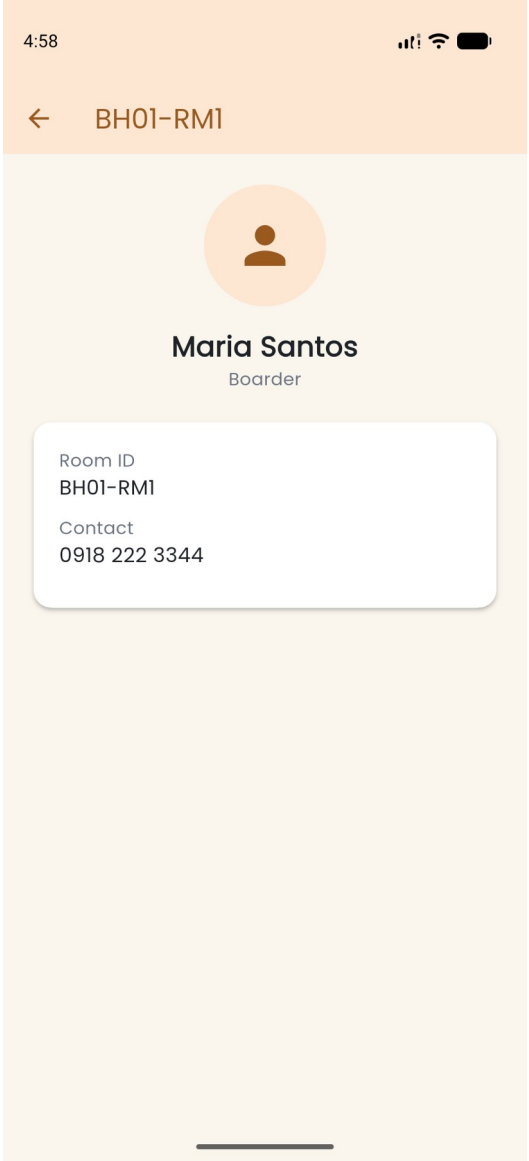


Figure 19. Read-only boardmate detail, October 26.

7.10 Owner profile and member details

OwnerProfileScreen displays the owner's contact information and supports editing the owner's name and contact number through AuthProvider. An owner opening a member receives a complete detail view. Shared InfoCard widgets present these fields consistently across profile destinations.

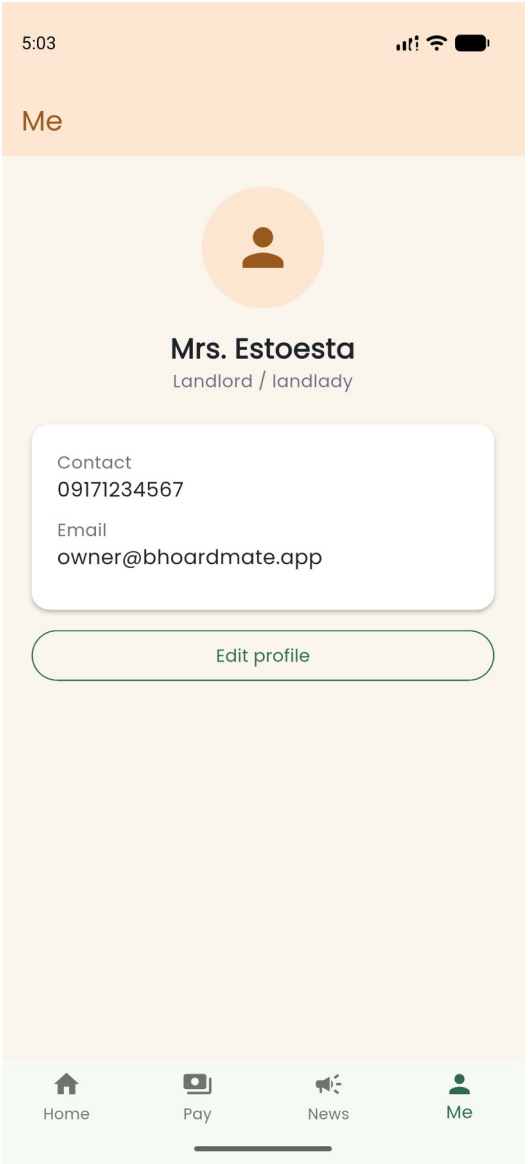


Figure 20. Owner Me profile, October 26.

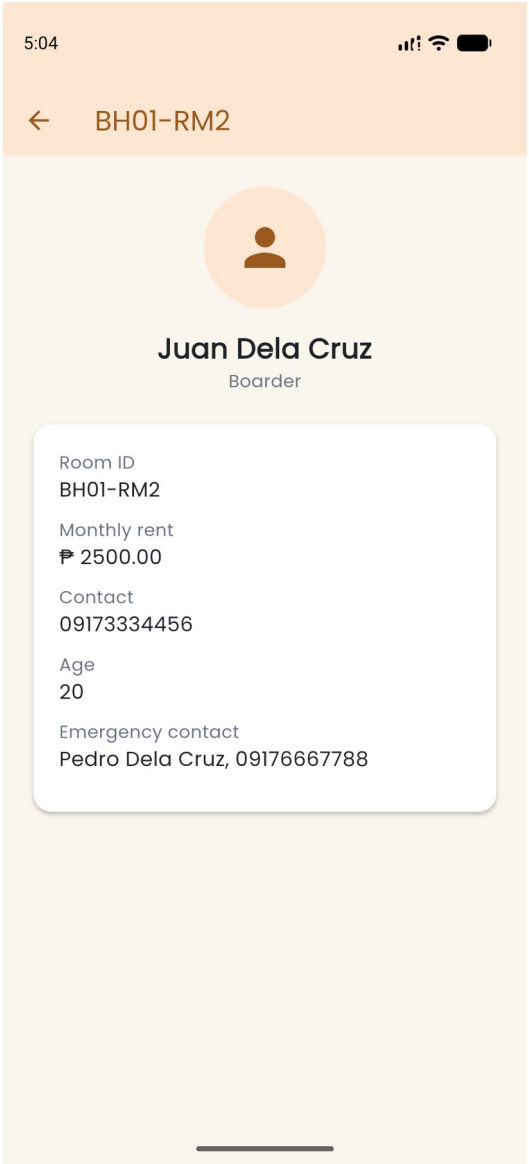


Figure 21. Member detail opened by the owner, October 26.

7.11 Owner room management

RoomsScreen is restricted to signed-in owners. It lists room identifiers and supports room creation, rent editing, and access-code reset. Resetting the code clears the departing boarder's profile while retaining the room identifier. The confirmation dialog makes the effect explicit before the owner proceeds. Figures use the project's seeded demonstration room records.

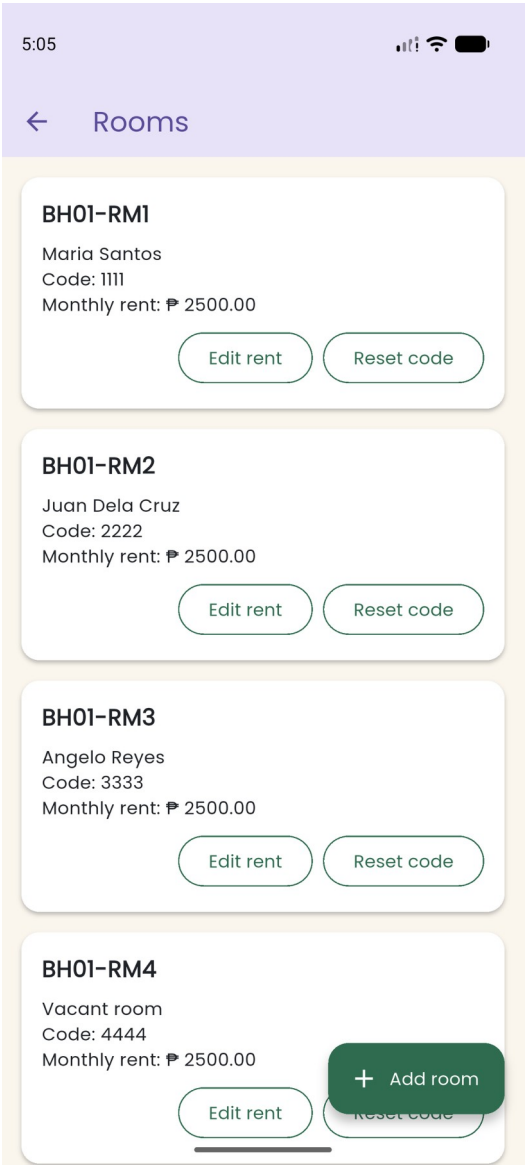


Figure 22. Owner room list with demonstration codes, October 26.

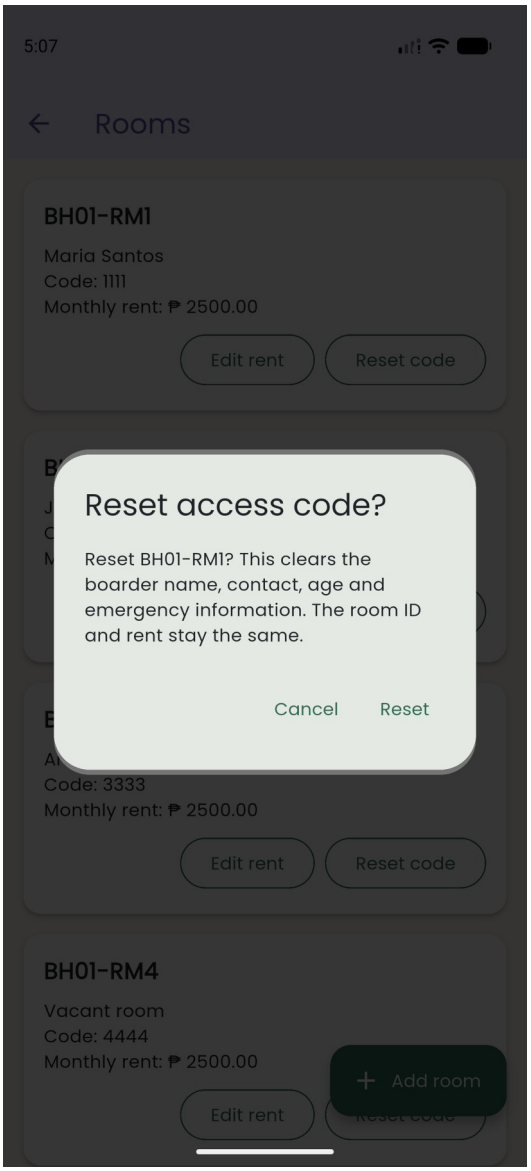


Figure 23. Confirmation before resetting room access, October 26.

7.12 Holidays, saved data and adjusted due dates

News displays Philippine public holidays obtained through HolidayService. If a request fails and a saved response is available, the UI displays the saved list with an offline notice. HolidayProvider also moves displayed due dates past weekends and known holidays. The original bill date stays in Firestore. A warning appears when the necessary holiday data has not loaded.

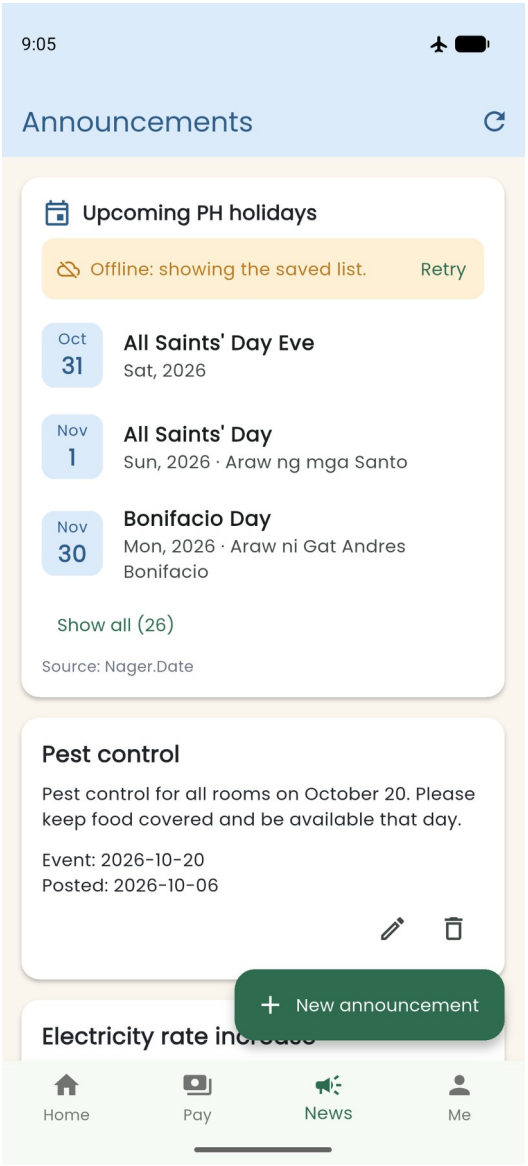


Figure 24. Saved holiday list and offline notice, October 25.

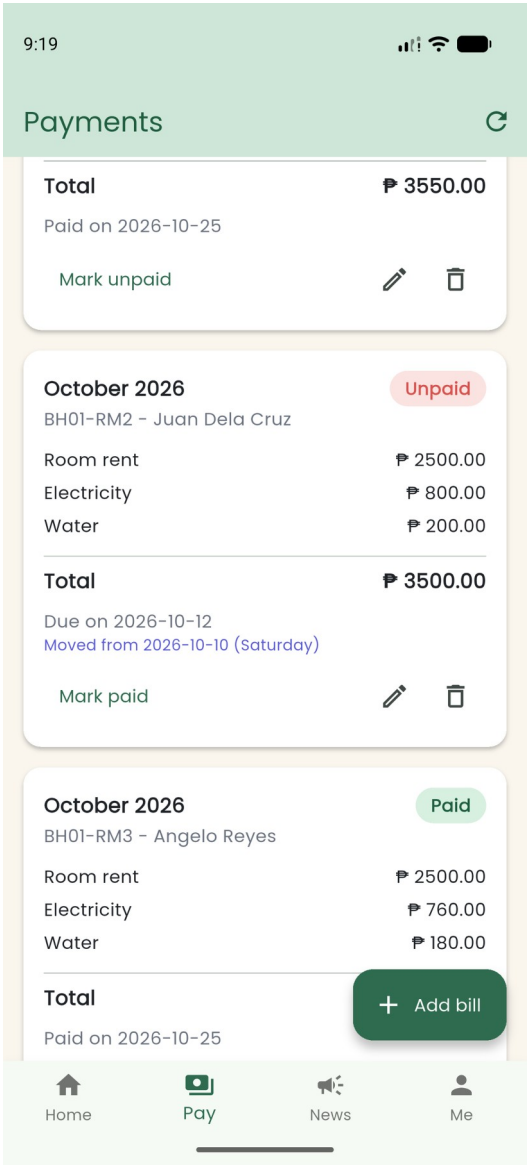


Figure 25. Adjusted displayed payment due date, October 25.

7.13 Validation and error feedback

Forms reject missing or invalid values before an action is sent to the provider. Profile and room fields have their own validation rules, and monetary inputs reject non-finite values such as NaN and Infinity. Screens also include loading, empty, and provider-error states, with Retry where appropriate.

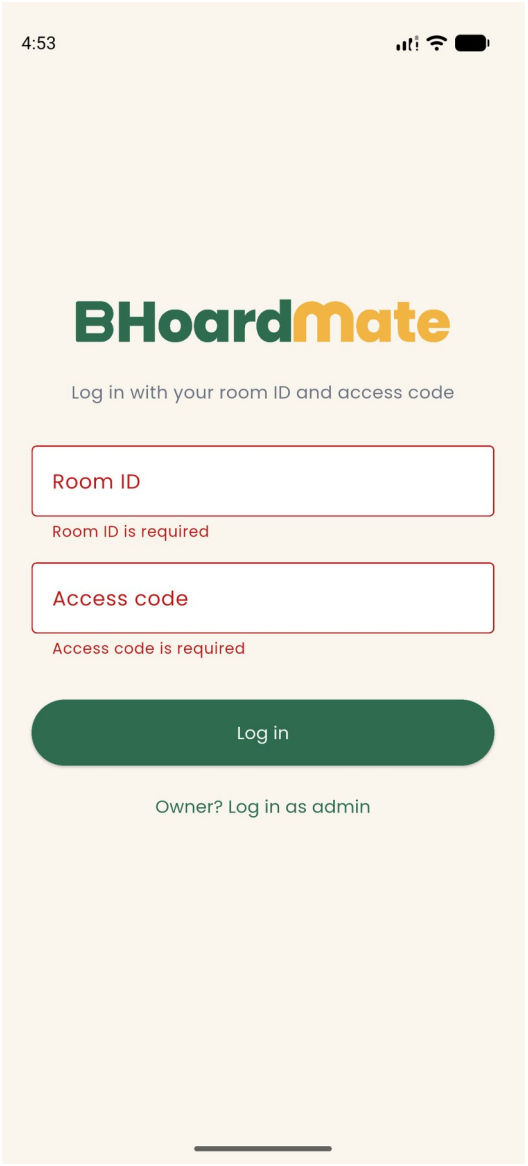


Figure 26. Login required-field errors, October 26.

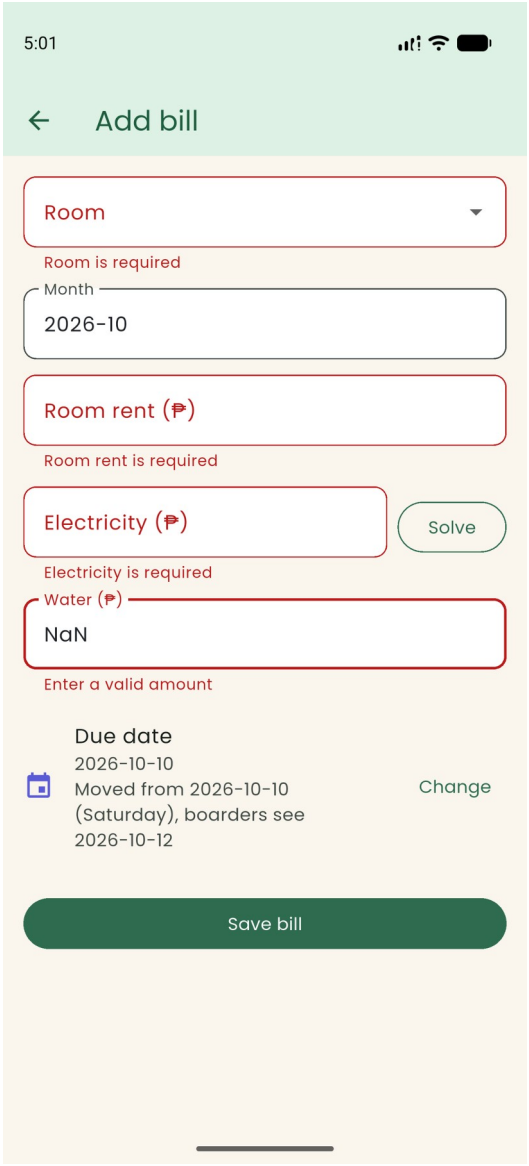


Figure 27. Bill-form validation errors, October 26.

7.14 Announcement and concern forms

The owner's announcement form requires a title and message, with an optional event date. Boarders report a maintenance concern using a title and description. The forms save through their respective providers.

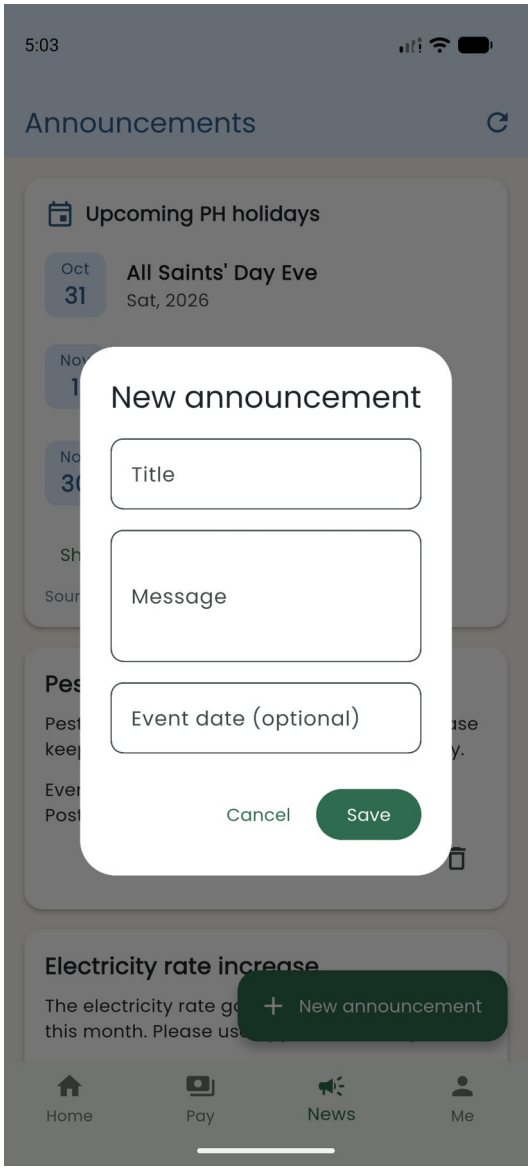


Figure 28. New-announcement form, October 26.

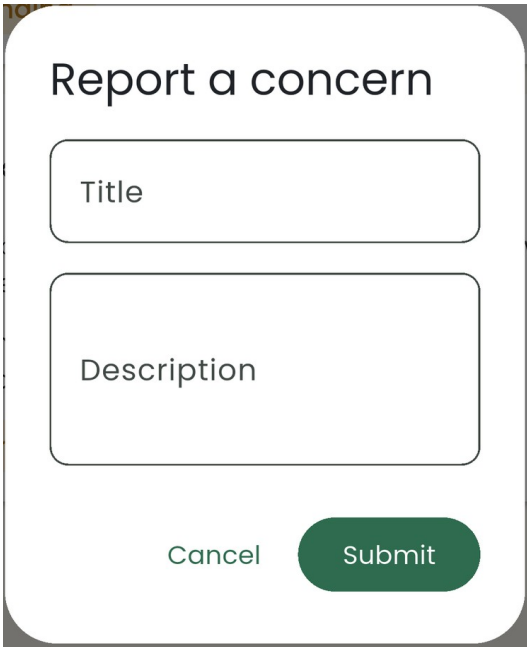


Figure 29. Report-concern dialog (cropped), October 26.

7.15 Profile editing and room creation

A boarder can edit the contact and emergency details of their own profile. The owner's Add Room form collects the room identifier, access code and monthly rent. Required fields and invalid values are checked before saving.

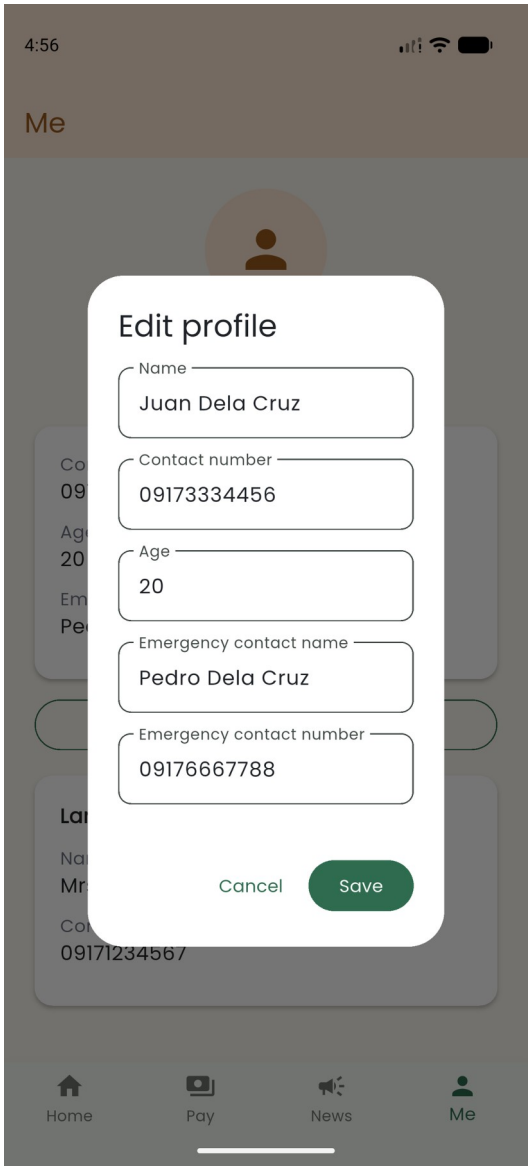


Figure 30. Own-profile editing form, October 26.

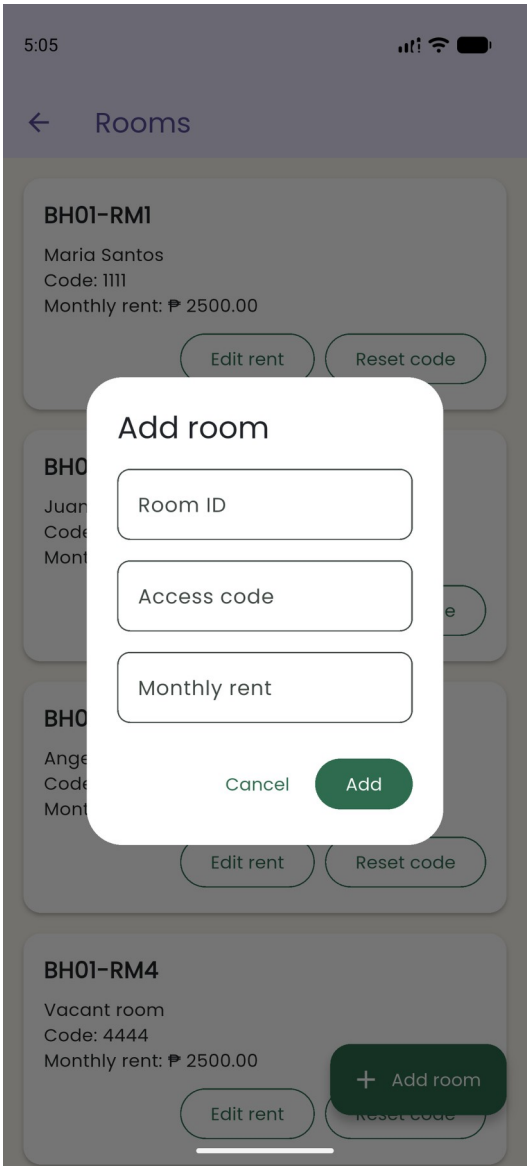


Figure 31. Owner Add Room form, October 26.

8. Technical Implementation

8.1 Navigation

What the app needed: after login, users move between four main areas (Home, Pay, News and Me), open extra pages such as Members, Concerns and Rooms, and come back with the Back button. Some pages also need to know which record was tapped, for example which room or which bill. We used Navigator 1.0 with `MaterialPageRoute` instead of named routes, because the app has only a few pages and most of them need a model passed in.

How it works: `lib/main.dart` starts Firebase, registers the providers and opens `SplashScreen`. After the animation, `lib/screens/splash_screen.dart` calls `Navigator.pushReplacement` to open `LoginScreen`. In `lib/screens/login_screen.dart`, `AuthProvider.loadSession()` checks for a saved session first. If there is one, the user goes straight to `MainScreen`; if not, they fill in the boarder or owner form. A successful login also uses `Navigator.pushReplacement`, so pressing Back cannot return to the splash or the login form.

`MainScreen` in `lib/screens/main_screen.dart` holds the four tabs in an `IndexedStack` with a `BottomNavigationBar`. The stack keeps every tab alive, so a tab keeps its state when the user switches away and back. `HomeScreen` gets an `onOpenTab` callback, so its Pay and News buttons change the same selected index as the bottom bar. From Home, `Navigator.push` opens Members, Concerns and, for the owner, Rooms. These pages sit on top of `MainScreen`, so Back closes them and returns to Home.

`MainScreen` also handles the phone's Back button with `PopScope(canPop: false)`. On Pay, News or Me, Back goes to Home. On Home, the first press shows a "Press back again to exit" SnackBar, and a second press within two seconds calls `SystemNavigator.pop()`. We added this after phone testing showed that Back on any tab closed the whole app, because `MainScreen` is the only route left after login (issue #54, pull request #55).

The Me tab is `MembersScreen(showOwnProfile: true)`. It shows `OwnerProfileScreen` for the owner and the boarder's own `MemberDetailScreen` for a boarder. The Members button on Home opens the same `MembersScreen` as a directory, so both entry points use the same profile widgets.

Passing data: in `lib/screens/members_screen.dart`, tapping a room opens `MemberDetailScreen(room: room)`. `lib/screens/member_detail_screen.dart` has a required `Room` parameter in its constructor. The screen uses the room ID to get the latest copy from `AuthProvider`, so later changes still show. In `lib/screens/payments_screen.dart`, tapping a bill opens `PaymentFormScreen(payment: payment)`. The `Payment` parameter is optional: when it is null the form adds a new bill, and when it has a value the form edits that bill and fills in its fields.

Without it: if login used a normal `Navigator.push`, Back would return to the login form while the user is still signed in. Without the `IndexedStack`, each tab would be rebuilt and lose its state every time the user switches tabs. Without constructor parameters, the detail and form screens would not know which room or bill to show. When the user logs out or a boarder's access is no longer valid, `MainScreen` clears the payment, announcement and concern providers and uses `Navigator.pushAndRemoveUntil` to go back to Login, so old pages cannot be reached with Back. [1, 4]

8.2 State Management and Data Access

What the app needed: several screens show the same data. The Home balance comes from the same bills as the Pay tab, and the signed-in role decides which buttons each screen shows. We used Provider with `ChangeNotifier`. The `MultiProvider` in `lib/main.dart` registers five providers: `AuthProvider` (role, room, owner and member records), `PaymentProvider` (bills), `AnnouncementProvider`, `ConcernProvider` and

HolidayProvider. Each provider also keeps its own loading and error values, so screens can show a spinner or an error message.

Which widgets use it: we use context.watch when a screen shows provider data and context.read inside button actions. HomeScreen watches AuthProvider, PaymentProvider, AnnouncementProvider and HolidayProvider to build the dashboard. PaymentsScreen watches PaymentProvider and HolidayProvider, AnnouncementsScreen and ConcernsScreen watch their own providers, and HolidaySection watches HolidayProvider. PaymentFormScreen uses context.read<PaymentProvider>() when saving a bill, and LoginScreen uses context.read<AuthProvider>() to log in. When a provider calls notifyListeners(), every widget that watches it rebuilds.

Data access: all Firebase code is in lib/services/database_service.dart. Screens never import Firestore; they call provider methods, and the providers call the service. Owners log in with Firebase email and password. Boarders log in with anonymous authentication, and a session document stores their room ID and access code. AuthProvider turns login errors into short messages for the login screen.

The service also gives snapshot streams. PaymentProvider.loadPayments loads the bills and then listens to DatabaseService.paymentsStream. The owner gets the bills of all rooms; a boarder only gets their own room's bills. When Firestore changes, the provider replaces its list and calls notifyListeners(). AnnouncementProvider and ConcernProvider work the same way. On logout, each provider cancels its stream, and a _loadCount number stops an old, slow response from bringing back data after logout.

CRUD example: PaymentFormScreen checks the form and calls addPayment or updatePayment. PaymentsScreen shows the list, marks a bill paid or unpaid with togglePaid, and asks for confirmation before deletePayment. PaymentProvider passes these calls to the database service. Payment in lib/models/payment.dart computes total from rent, electricity and water. fromMap takes the Firestore document ID separately, and toMap returns only the stored fields.

Without it: each screen would load its own copy of the bills. If the owner marks a bill paid on the Pay tab, Home would still show the old balance until it reloaded, and we would have to pass data and callbacks through many constructors. Security is handled separately: Firestore rules only let a boarder edit the allowed profile fields of their own room, and access codes are kept in the owner-only roomCodes collection. The profile screen also hides other boarders' rent, age and emergency details. [1, 5, 6, 9, 10]

8.3 REST API, Offline Data and Due Dates

What the app needed: rent due dates should not fall on a public holiday, and boarders want to see upcoming holidays in News. We used the Nager.Date API. HolidayService in lib/services/holiday_service.dart uses the http package to send a GET request to https://date.nager.at/api/v3/PublicHolidays/{year}/PH. It checks the status code and has a 10-second timeout. A network error, a bad status code or invalid JSON becomes a HolidayException with a clear message.

Sample JSON and model: the API returns a JSON list. On October 26, 2026, the 2026/PH request returned entries such as date: 2026-01-01, localName: Bagong Taon and name: New Year's Day; the sample below keeps only the three fields we use [11]. Holiday.fromJson in lib/models/holiday.dart turns date into a DateTime and reads localName and name as strings. With a model class, the rest of the code uses holiday.date instead of reading raw JSON keys everywhere.

Saved data: after a successful request, HolidayService saves the response in shared_preferences under a key like holidays_2026. HolidayProvider.loadHolidays loads last year, this year and next year separately. If one year fails, it uses that year's saved copy, and the other years still work. The provider keeps the loading flag, the error message, whether saved data was used, and which years are available.

Loading, error and success states: the News screen shows HolidaySection from lib/widgets/holiday_section.dart. It shows a spinner while loading, the holiday list on success, an error

card with a Retry button when nothing loads, and an offline notice when saved data is shown. It also shows a message when there are no upcoming holidays.

Due dates: `HolidayProvider` also has `dayOffReason`, `nextWorkingDay`, `dueDate` and `dueDateNote`. They move a displayed due date past weekends and holidays and add a short note. Home, Payments and the payment form show this adjusted date, but the original date stays in Firestore. If the holiday data for that year did not load, the app warns that only weekends were checked. Our automated tests cover these rules.

Without it: we would have to type the holiday list into the code by hand and update it every year. Due dates would only skip weekends, so rent could still fall on a holiday such as Rizal Day. Without the saved copy, a failed request would leave the holiday list empty. [1, 7, 8, 11]

Sample API response — fetched October 26, 2026:


```
[
  {
    "date": "2026-01-01",
    "localName": "Bagong Taon",
    "name": "New Year's Day"
  },
  {
    "date": "2026-02-17",
    "localName": "Chinese New Year",
    "name": "Chinese New Year"
  }
]
```

Current navigation map

The figure below shows the navigation of the final app, including the separate Members directory and the Me tab. [1]

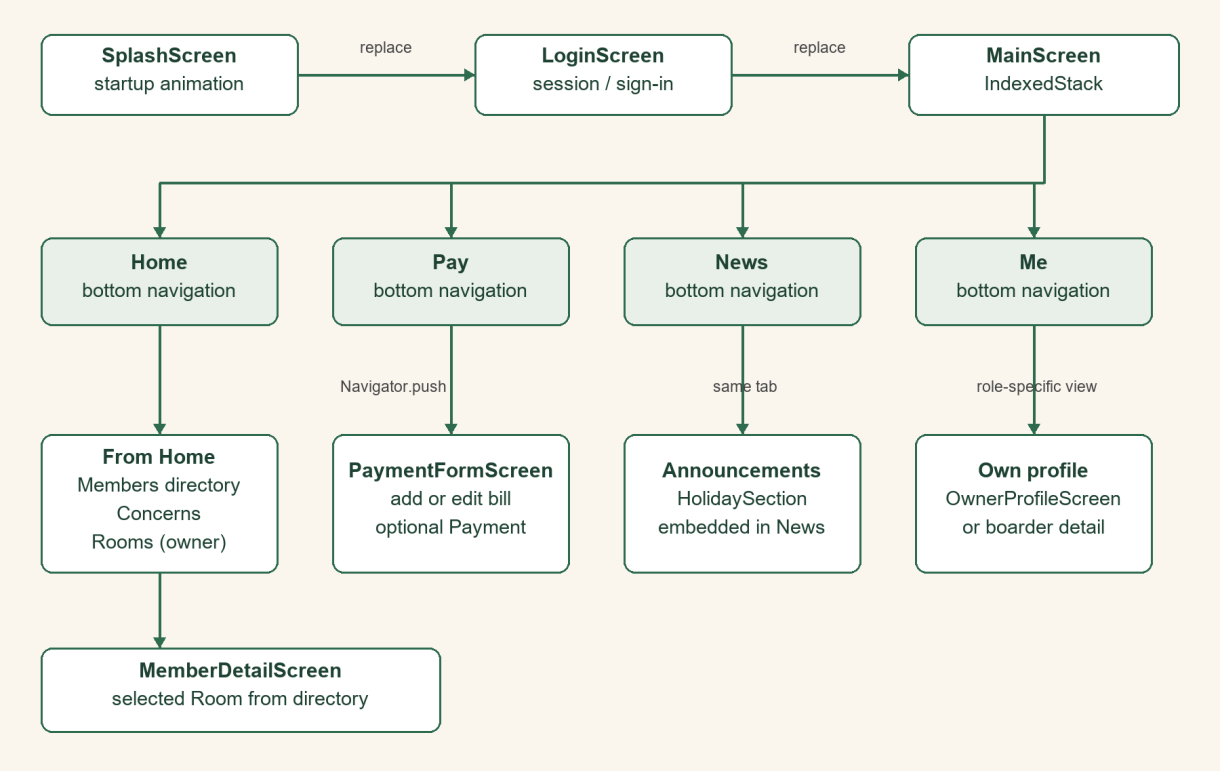


Figure 32. Current navigation and data-passing map derived from the source, October 26, 2026.

8.4 Models, separation of concerns and reusable widgets

We split the code into five folders: `models`, `providers`, `services`, `screens` and `widgets`. Models turn Firestore and JSON data into Dart objects. Providers hold shared state. Services talk to Firebase and the holiday API. Screens only call provider methods, so no screen has Firestore or HTTP code. The packages we use are listed in `pubspec.yaml`: provider, http, shared_preferences, firebase_core, cloud_firestore, firebase_auth, google_fonts and cupertino_icons, plus flutter_launcher_icons, flutter_native_splash and flutter_lints for development. [1, 6–8, 14–21]

Model/state	Contract used by the implementation
Room	Room-code string ID; rent and optional profile/contact fields. Access codes are kept outside shared room documents.
Owner	Firebase UID, name, email, and contact number; password is handled by Firebase Authentication.
Payment	Nullable string document ID, roomId, month, rent/electricity/water, dueDate, isPaid, and paidDate; total is a getter.
Announcement / Concern	Nullable string document IDs. Announcements store title/message/dates; concerns also store roomId and pending/done status.
Holiday	DateTime date, localName, and name parsed from JSON. Saved API responses provide the per-year fallback.

Reusable widget	Used in more than one screen
BalanceCard	HomeScreen and PaymentsScreen
StatusChip	PaymentsScreen and ConcernsScreen
InfoCard	MemberDetailScreen and OwnerProfileScreen
EmptyState	Members, Rooms, Announcements, and other empty-list views

8.5 Testing, debugging and release evidence

On October 27 we ran the checks on main 852ac75 (v1.0.1): `dart format` changed 0 of 56 files, `flutter analyze` found no issues, and all 86 tests passed. The tests use fake database and holiday services, so they run without Firebase or the internet. [1, 21, 22]

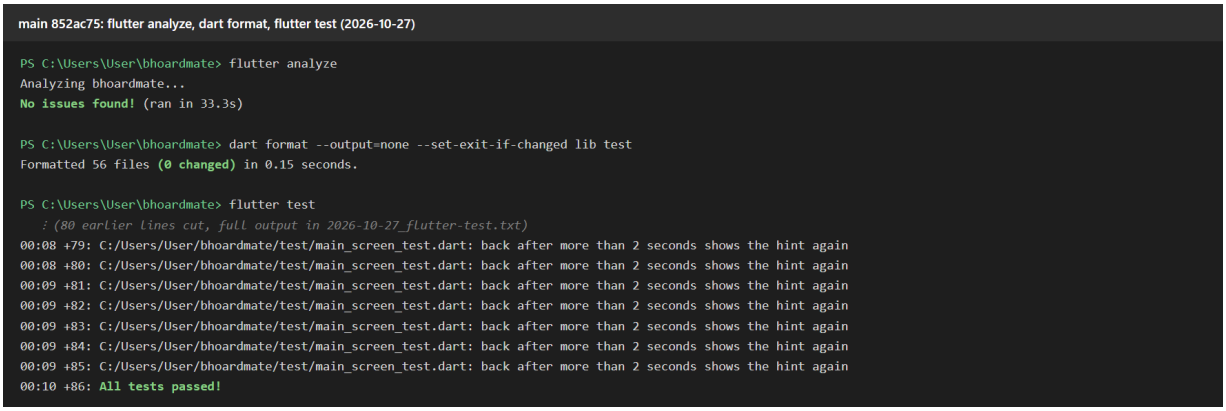


Figure 33. Formatting, analyzer and automated-test results on main 852ac75, October 27.

Documented debugging case: duplicate Hero tags

Debugging case: during pull request #45 (Payments), the owner-login test crashed when it opened `MainScreen`. Payments and News each have a `FloatingActionButton`, and both stay mounted in the `IndexedStack`. Both buttons used the same default Hero tag. [1, 23]

The error said “There are multiple heroes that share the same tag within a subtree”, and the stack trace pointed to `Hero._allHeroesFor`. We fixed it by setting `heroTag: null` on the Payments button, because it does not need a Hero animation. On October 26 we reproduced the crash again and then showed the six login tests passing with the fix (Figures 34 and 35). [1, 23]

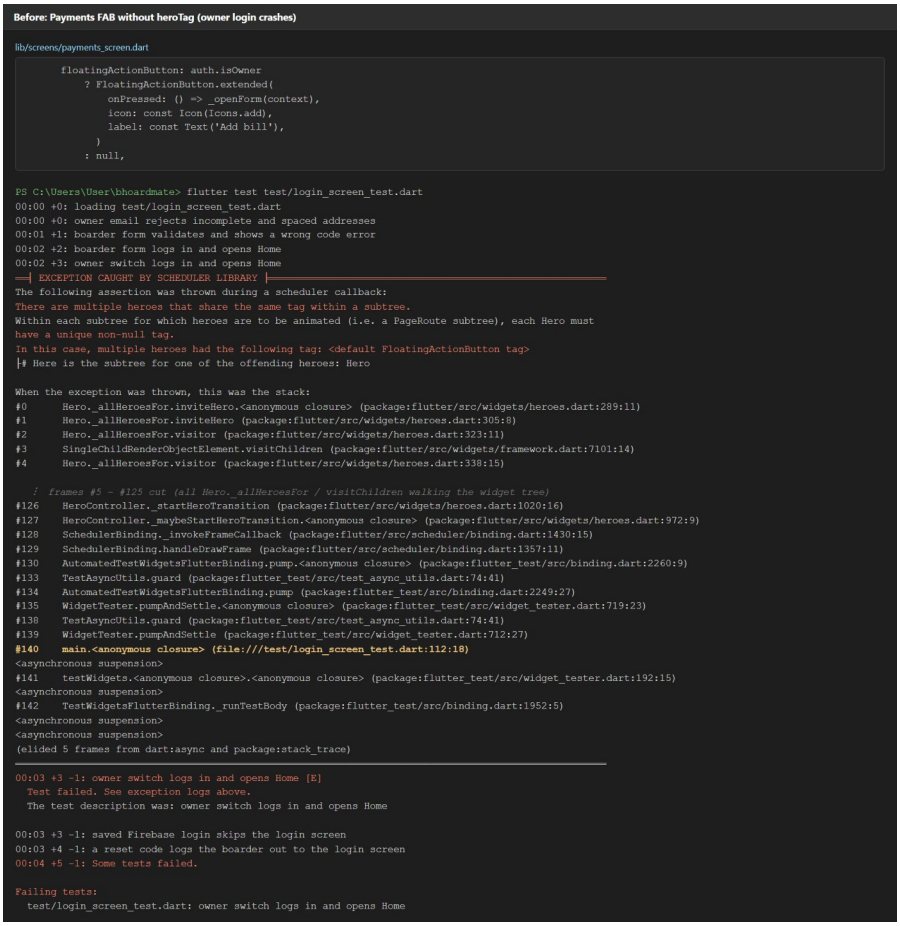


Figure 34. Reproduction of the earlier duplicate-Hero assertion with the owner-login test, October 26.

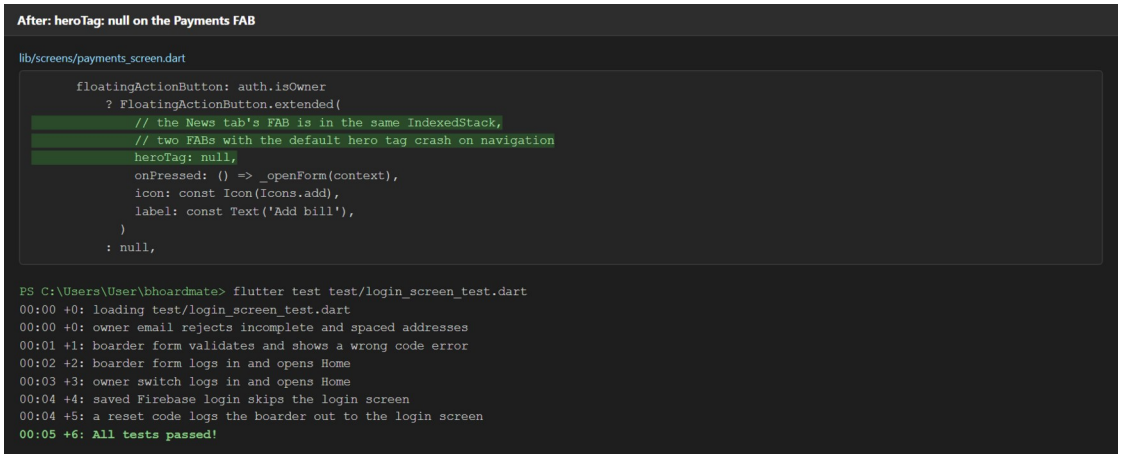


Figure 35. Recorded fix and six passing login tests, October 26.

Full text evidence: docs/screenshots/debugging/2026-10-26_hero-tag-crash_before.txt and the matching after.txt file. The v1.0.1 test run is saved in docs/screenshots/tooling/2026-10-27_flutter-test.txt. [1]

Build procedure and observed results

We built v1.0.1 with Flutter 3.44.5 and Dart 3.12.2, running these commands in order: [1]

```
flutter pub get
dart format --output=none --set-exit-if-changed lib test
flutter analyze
flutter test
flutter build apk --debug
flutter build apk --release
```

Recorded result	Evidence / limitation
Debug APK	Built: 160,763,642 bytes (153.32 MiB). Not installed; the release APK is the build given to users.
Release APK	Built: 55,462,731 bytes (52.89 MiB), 65.5% smaller than debug. The published v1.0.1 APK was installed and checked on the Medium Phone API 36.1 emulator: Back from Pay returns to Home, the first Back on Home shows the exit hint, and the second exits. [1, 27]
Identity	com.bj.bhoardmate; BHoardMate; versionName 1.0.1 / versionCode 2; minimum API 24; target API 36.
Signing / mode	Both APKs are signed with the Android debug certificate; we did not make a separate release key.
Debug vs release	Debug keeps debugging and hot reload support, so it is much larger. Release is compiled ahead of time and optimized. Both hide the DEBUG banner by configuration.

Debug output: build/app/outputs/flutter-apk/app-debug.apk. Release output: build/app/outputs/flutter-apk/app-release.apk. [1]

We published v1.0.1 on GitHub with the BHoardMate-v1.0.1.apk file, built from main 852ac75. The older v1.0.0 release (89315cf) is still on the releases page. [26, 27]

Published v1.0.1 APK SHA-256:
`0fcaeb668c5321474898388c2c2d7eb32db24a6eac2b8c335f77b48e53a40bf9`

The POCO F5 Pro test recorded in pull request #55 checked the back-button fix before the version bump. It is separate from the published v1.0.1 emulator check shown in Figures 36–39. [1, 28]

Published v1.0.1: Back from Pay to Home

On October 29, the published release APK was checked on the Medium Phone API 36.1 emulator (1080 × 2400, portrait, light mode), signed in as demo boarder BH01-RM2. These screenshots show Pay before Back and Home after one Back press, without the exit hint. Pull request #58 preserves the images and installed-version record. [1, 27]

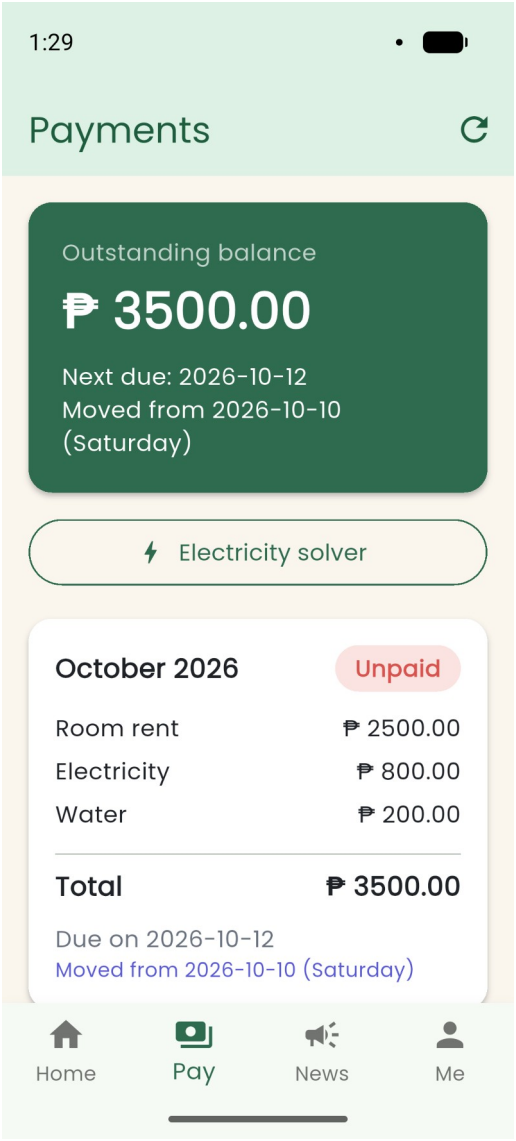


Figure 36. Published v1.0.1 on Pay before pressing Back, October 29.

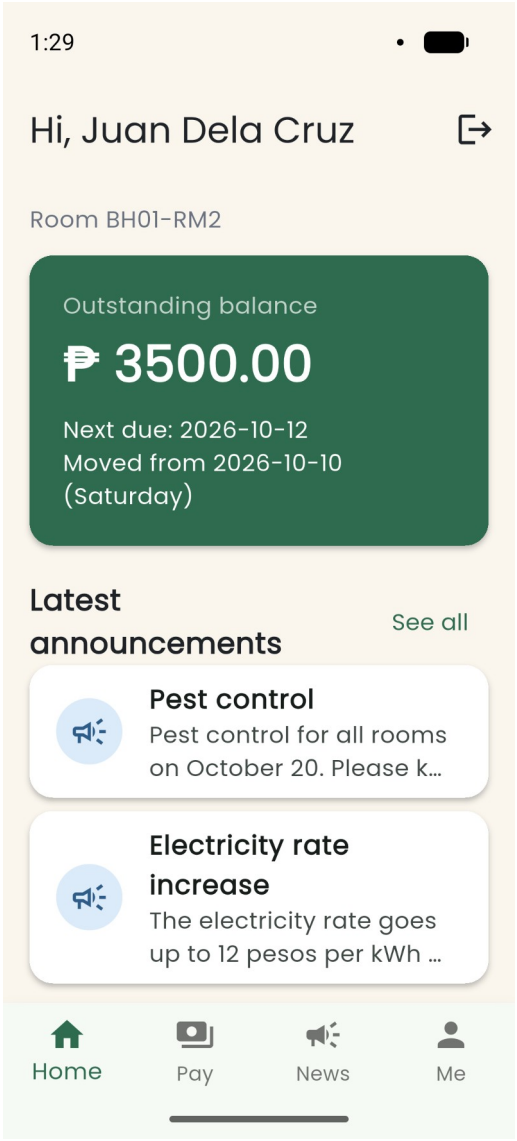


Figure 37. One Back press from Pay returns to Home without the exit hint, October 29.

Published v1.0.1: Double Back to exit

On Home, the first Back press displays “Press back again to exit.” A second Back press within two seconds leaves the app and reveals the Android launcher. The published APK hash and size match the installed-build record in pull request #58. [1, 27]

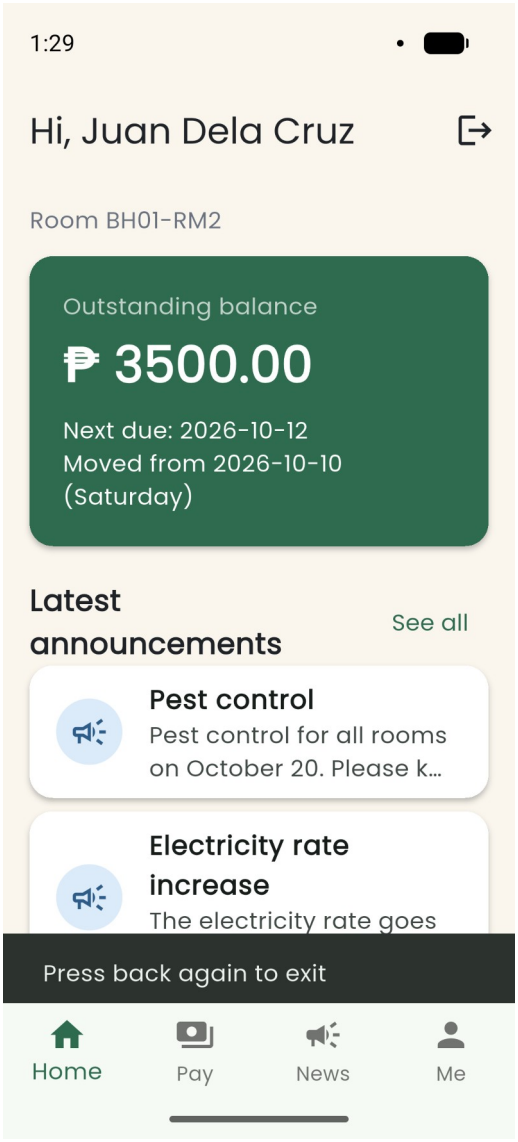


Figure 38. First Back press on Home shows the exit hint, October 29.

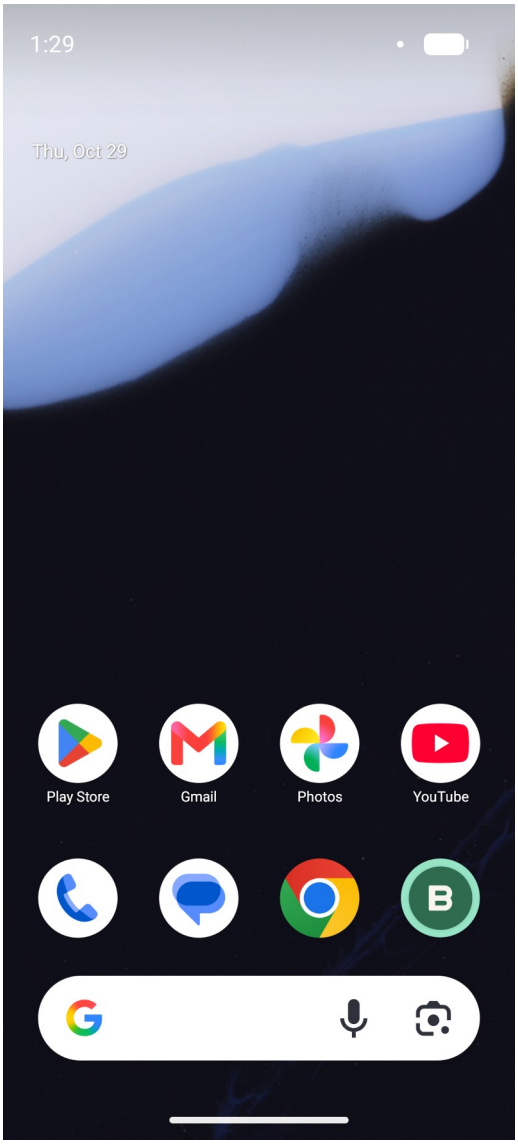


Figure 39. Second Back press leaves the app; the emulator launcher is visible, October 29.

8.6 GitHub workflow and traceable review

We worked on feature branches and merged through pull requests. Each task was a GitHub issue on the project board. Pull requests #45, #51 and #52 had requested changes or specific feedback, then fixes and approval. Pull request #55 fixed a bug we found while testing on a phone. [1, 23–25, 28]

Pull request	Concrete review evidence
#45 — Payments	BryWick requested preservation of cents, finite numeric validation and visible refresh failures. Johnmarc recorded the corrections and Bryan approved. [23]
#51 — Icon / splash	Johnmarc requested login wordmark alignment after emulator testing and later approved the revised layout and tests. [24]
#52 — README / evidence	Johnmarc requested the demo boarder login in Setup and noted the reproduced Hero-failure evidence. The PR was revised and merged. [25]
#55 — Back button	Johnmarc fixed the system Back button closing the app (#54) after phone testing. BryWick approved it and merged it. [28]

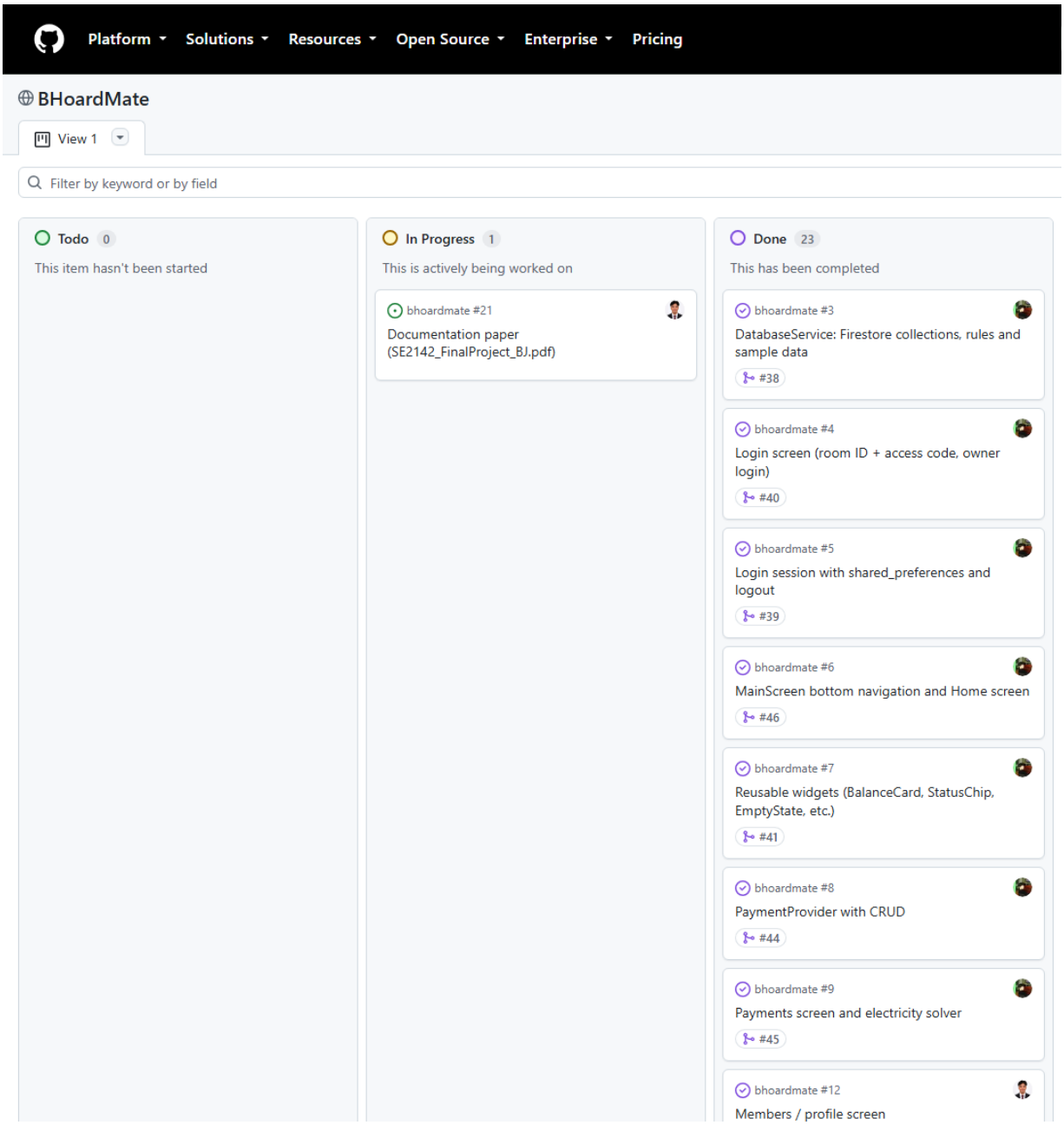


Figure 40. GitHub Projects board (top part): Todo 0, In Progress 1 (#21), Done 23, October 27.

9. References

Package versions are the versions locked in pubspec.lock for v1.0.1.

[1] Group BJ, “BHoardMate: source code, README and project evidence,” 2026. [Source repository]. Release app source: 852ac75fb0a84322c8402bd900fb30178b22150b. Additional evidence: merged pull requests #57 (October 27 tooling and project board) and #58 (October 29 v1.0.1 emulator check).

Available: https://github.com/ohimeee/bhoardmate/tree/852ac75fb0a84322c8402bd900fb30178b22150b. Accessed: 2026-10-29.

Evidence PR #57: https://github.com/ohimeee/bhoardmate/pull/57. Accessed: 2026-10-29.

Evidence PR #58: https://github.com/ohimeee/bhoardmate/pull/58. Accessed: 2026-10-29.

[2] SE2142 course materials, “FINAL PROJECT 2026,” 2026. [Course handout]. Unpublished course handout.

[3] B. K. F. Estoesta and J. B. Jayme (Group BJ), “ACTIVITY 1 — Group Formation and Project Proposal,” 2026. [Project proposal]. Completed Group BJ project proposal.

[4] Flutter, “Navigate to a new screen and back,” n.d. [Online documentation].

Available: https://docs.flutter.dev/cookbook/navigation/navigation-basics. Accessed: 2026-10-26.

[5] Flutter, “Simple app state management,” n.d. [Online documentation].

Available: https://docs.flutter.dev/data-and-backend/state-mgmt/simple. Accessed: 2026-10-26.

[6] R. Rousselet and contributors, “provider,” n.d. [Software package]. Version 6.1.5+1.

Available: https://pub.dev/packages/provider. Accessed: 2026-10-26.

[7] Dart project, “http,” n.d. [Software package]. Version 1.6.0.

Available: https://pub.dev/packages/http. Accessed: 2026-10-26.

[8] Flutter, “shared_preferences,” n.d. [Software package]. Version 2.5.6.

Available: https://pub.dev/packages/shared_preferences. Accessed: 2026-10-26.

[9] Firebase, “Get realtime updates with Cloud Firestore,” n.d. [Online documentation].

Available: https://firebase.google.com/docs/firestore/query-data/listen. Accessed: 2026-10-26.

[10] Firebase, “Authenticate with Firebase anonymously (Flutter),” n.d. [Online documentation].

Available: https://firebase.google.com/docs/auth/flutter/anonymous-auth. Accessed: 2026-10-26.

[11] Nager.Date contributors, “Nager.Date: public-holiday API and source,” n.d. [API and source repository]. Philippine public-holiday API; response accessed October 26, 2026.

Available: https://github.com/nager/Nager.Date. Accessed: 2026-10-26.

[12] OpenAI, “Codex,” 2026. [AI tool]. Used for repository inspection, drafting and document preparation.

Available: https://openai.com/codex/. Accessed: 2026-10-26.

[13] Anthropic, “Claude Code,” 2026. [AI tool]. AI assistance used by the group.

Available: https://code.claude.com/docs/en/overview. Accessed: 2026-10-27.

[14] Flutter, “google_fonts,” n.d. [Software package]. Version 8.2.1.

Available: https://pub.dev/packages/google_fonts. Accessed: 2026-10-26.

[15] Flutter, “cupertino_icons,” n.d. [Software package]. Version 1.0.9.

Available: https://pub.dev/packages/cupertino_icons. Accessed: 2026-10-26.

[16] Firebase, “firebase_core,” n.d. [Software package]. Version 5.1.0.

Available: https://pub.dev/packages/firebase_core. Accessed: 2026-10-26.

[17] Firebase, “cloud_firestore,” n.d. [Software package]. Version 7.1.0.

Available: https://pub.dev/packages/cloud_firestore. Accessed: 2026-10-26.

[18] Firebase, “firebase_auth,” n.d. [Software package]. Version 7.0.1.

Available: https://pub.dev/packages/firebase_auth. Accessed: 2026-10-26.

[19] Flutter Community, “flutter_launcher_icons,” n.d. [Software package]. Version 0.14.4.

Available: https://pub.dev/packages/flutter_launcher_icons. Accessed: 2026-10-26.

[20] J. Hanson and contributors, “flutter_native_splash,” n.d. [Software package]. Version 2.4.8.

Available: https://pub.dev/packages/flutter_native_splash. Accessed: 2026-10-26.

[21] Flutter, “flutter_lints,” n.d. [Software package]. Version 6.0.0.

Available: https://pub.dev/packages/flutter_lints. Accessed: 2026-10-26.

[22] Flutter, “flutter_test library,” n.d. [API documentation].

Available: https://api.flutter.dev/flutter/flutter_test/. Accessed: 2026-10-26.

[23] Group BJ, “PR #45: Payments screen review and recorded corrections,” 2026. [Pull request review].

Available: https://github.com/ohimeee/bhoardmate/pull/45#pullrequestreview-5638413243. Accessed: 2026-10-26.

[24] Group BJ, “PR #51: App icon and splash review,” 2026. [Pull request review].

Available: https://github.com/ohimeee/bhoardmate/pull/51#pullrequestreview-5643976965. Accessed: 2026-10-26.

[25] Group BJ, “PR #52: README and evidence review,” 2026. [Pull request review].

Available: https://github.com/ohimeee/bhoardmate/pull/52#pullrequestreview-5654634133. Accessed: 2026-10-26.

[26] Group BJ, “BHoardMate v1.0.0,” 2026. [Software release]. Published October 26, 2026; app source 89315cf.

Available: https://github.com/ohimeee/bhoardmate/releases/tag/v1.0.0. Accessed: 2026-10-27.

[27] Group BJ, “BHoardMate v1.0.1,” 2026. [Software release]. Published October 27, 2026; app source 852ac75.

Available: https://github.com/ohimeee/bhoardmate/releases/tag/v1.0.1. Accessed: 2026-10-27.

[28] Group BJ, “PR #55: Back button: go to Home first, double back to exit,” 2026. [Pull request].

Available: https://github.com/ohimeee/bhoardmate/pull/55. Accessed: 2026-10-27.